GOSIM

GOSIM Shenzhen 2026

KV Caching for Agentic Inference

From RadixAttention to Unified Radix Cache

Yichi Zhang

RadixArk Member of Technical Staff

SGL



CONTENTS

GOSIM Shenzhen 2026

01	**Workloads**	New challenges in agentic inference
02	**RadixAttention**	Reuse shared prefixes
03	**HiCache**	Retain and restore KV across tiers
04	**Unified Radix Tree**	Compose different state types
05	**Real-Workload Results**	Measured results and next steps



SGL

GOSIM

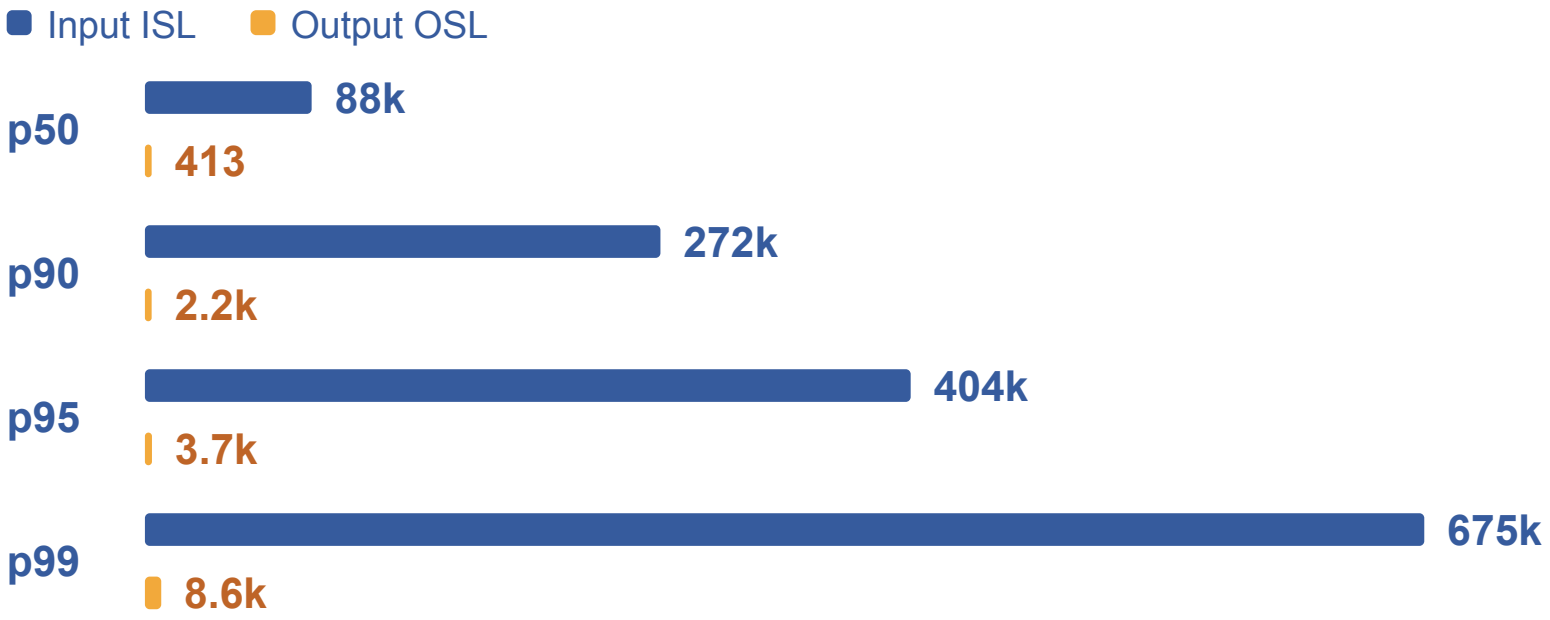
GOSIM Shenzhen 2026

01 Workloads



Agent Workloads: Long Inputs, Short Outputs

Measured ISL / OSL distributions · 393 real traces



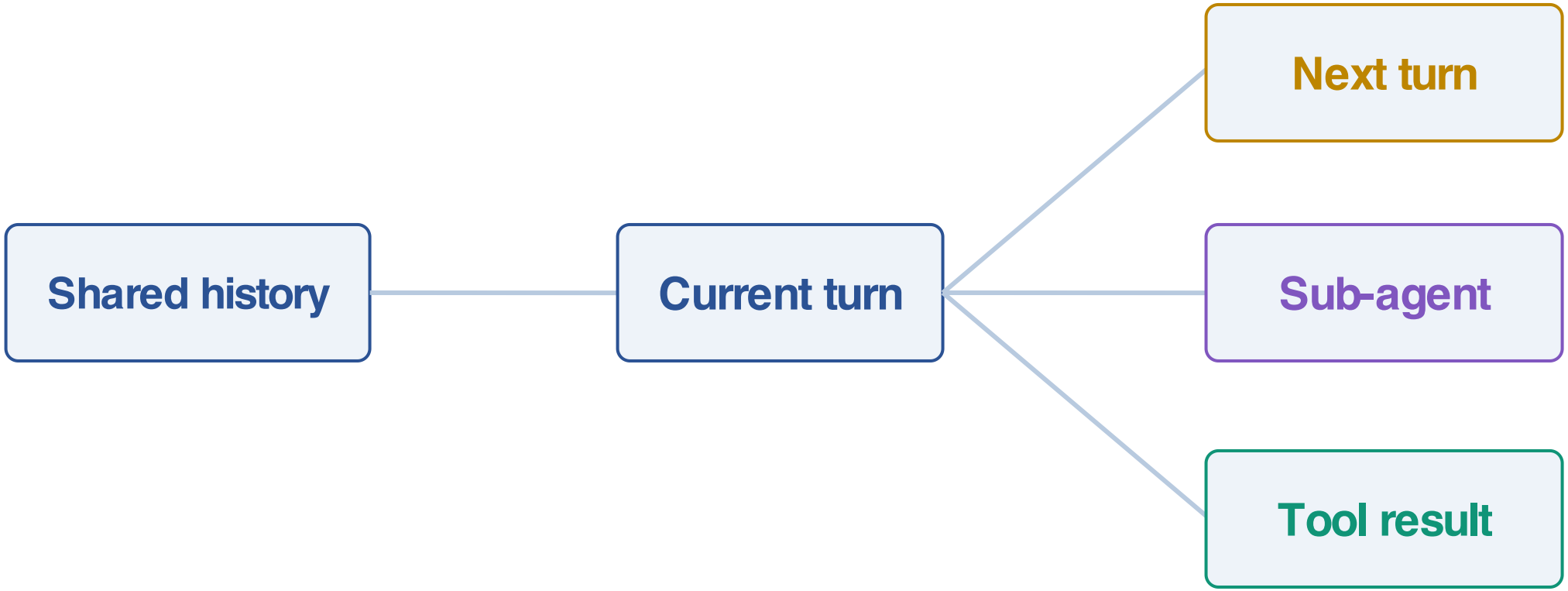
213 : 1

p50 input / output

Main cost: repeatedly processing historical context

Source: SemiAnalysis AgentX · inferencex.semianalysis.com

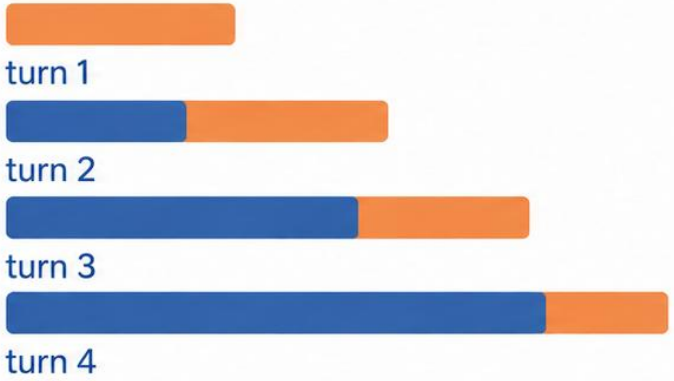
How Agents Reuse Context



How Agents Reuse Context

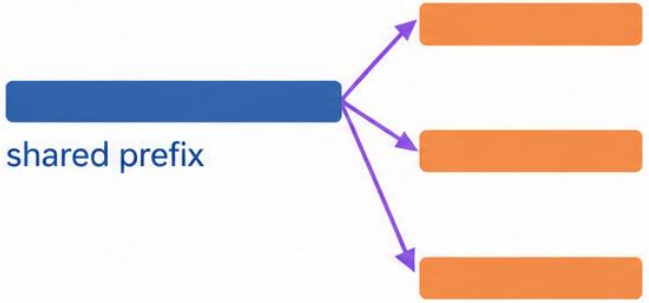
Reused history prefix New tokens this turn

Multi-turn Appends



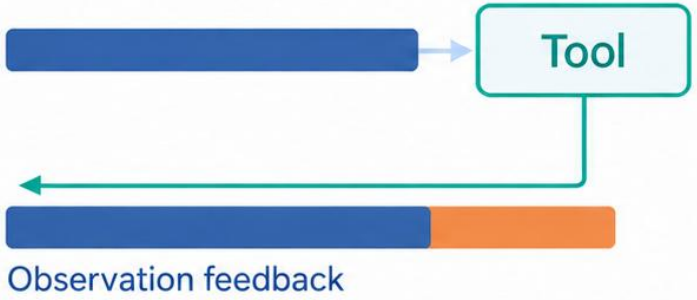
Context only grows;
each turn carries the full history.

Sub-agent Branching



All branches share the same prefix
up to the branching point.

Tool Loops / Retries



Reuse prior context;
append only new results.

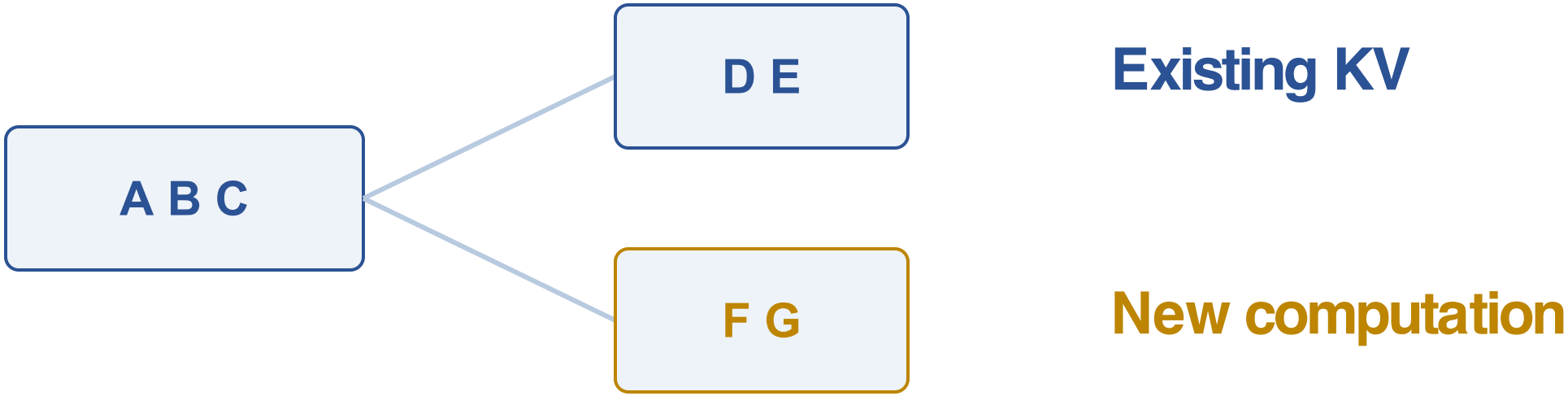
02

RadixAttention



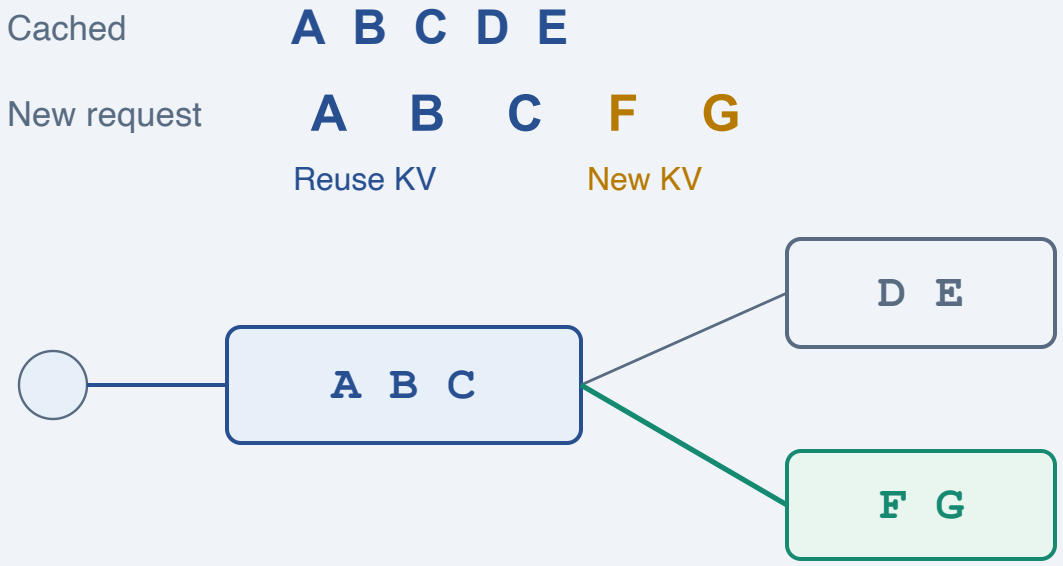
RadixAttention: KV Reuse by Token Prefix

Reuse the matching prefix; prefill only the new suffix



RadixAttention: A Partial Prefix Hit

Request A B C F G reuses KV for A B C



Share ABC; cache each suffix separately

3 Insert a branch

Add an F G branch

One copy of ABC

Reuse for later requests

3 reused **2 new**

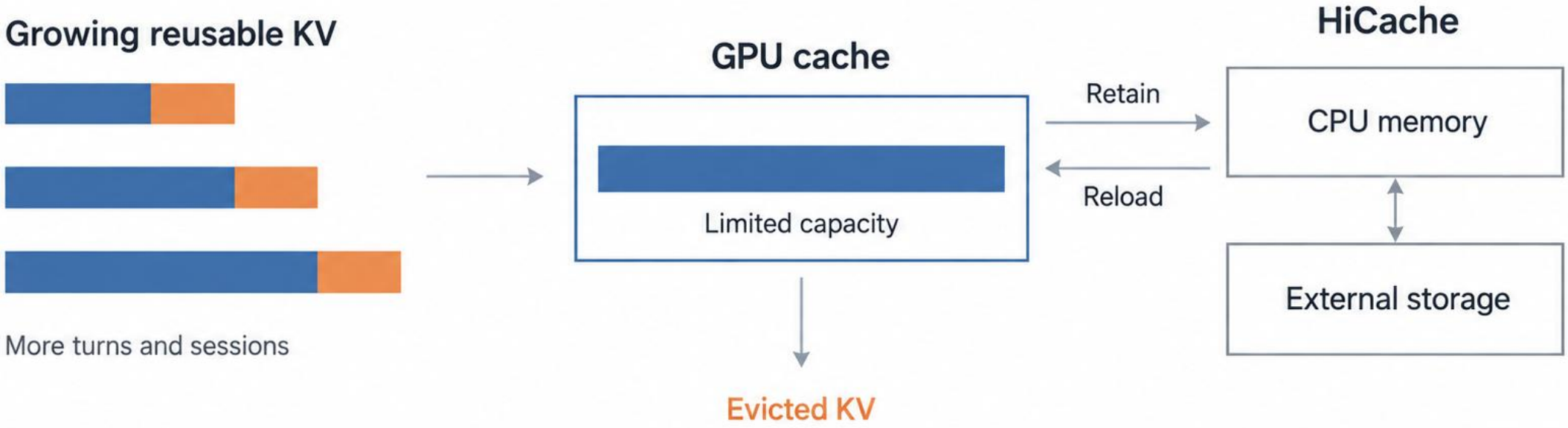
Cache one copy of the shared prefix; compute only new tokens.

03

HiCache



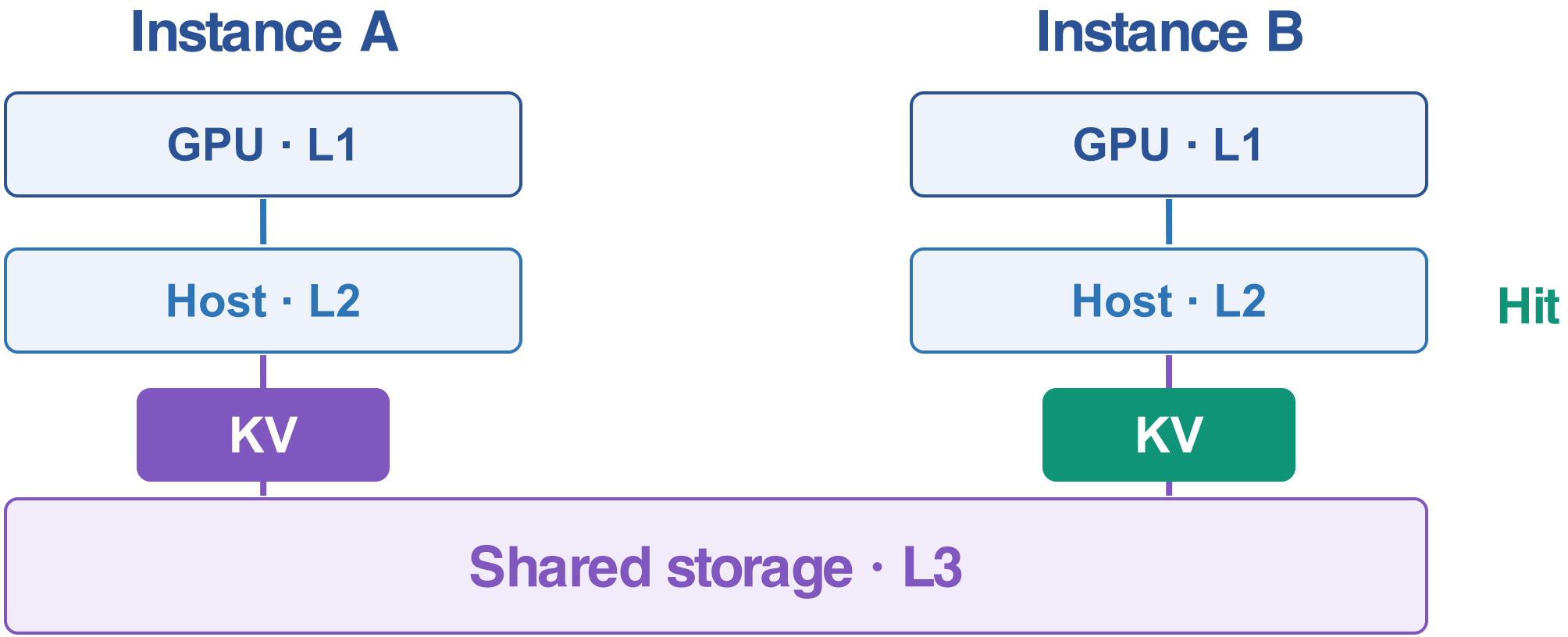
HiCache: The GPU Cache Capacity Limit



Without retained KV, the next turn may need prefill again.

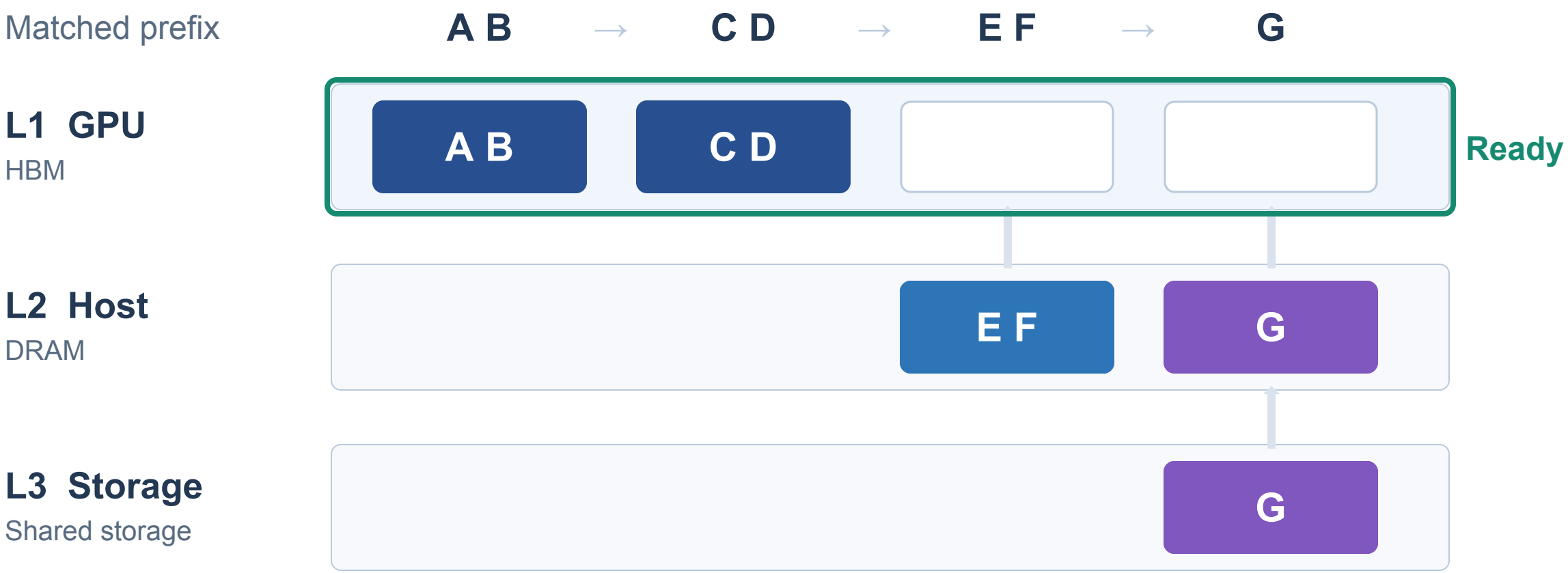
HiCache: Cache Reuse Across Instances via L3

Instance B matches the same prefix and restores it



HiCache: Bringing Cached KV Back to the GPU

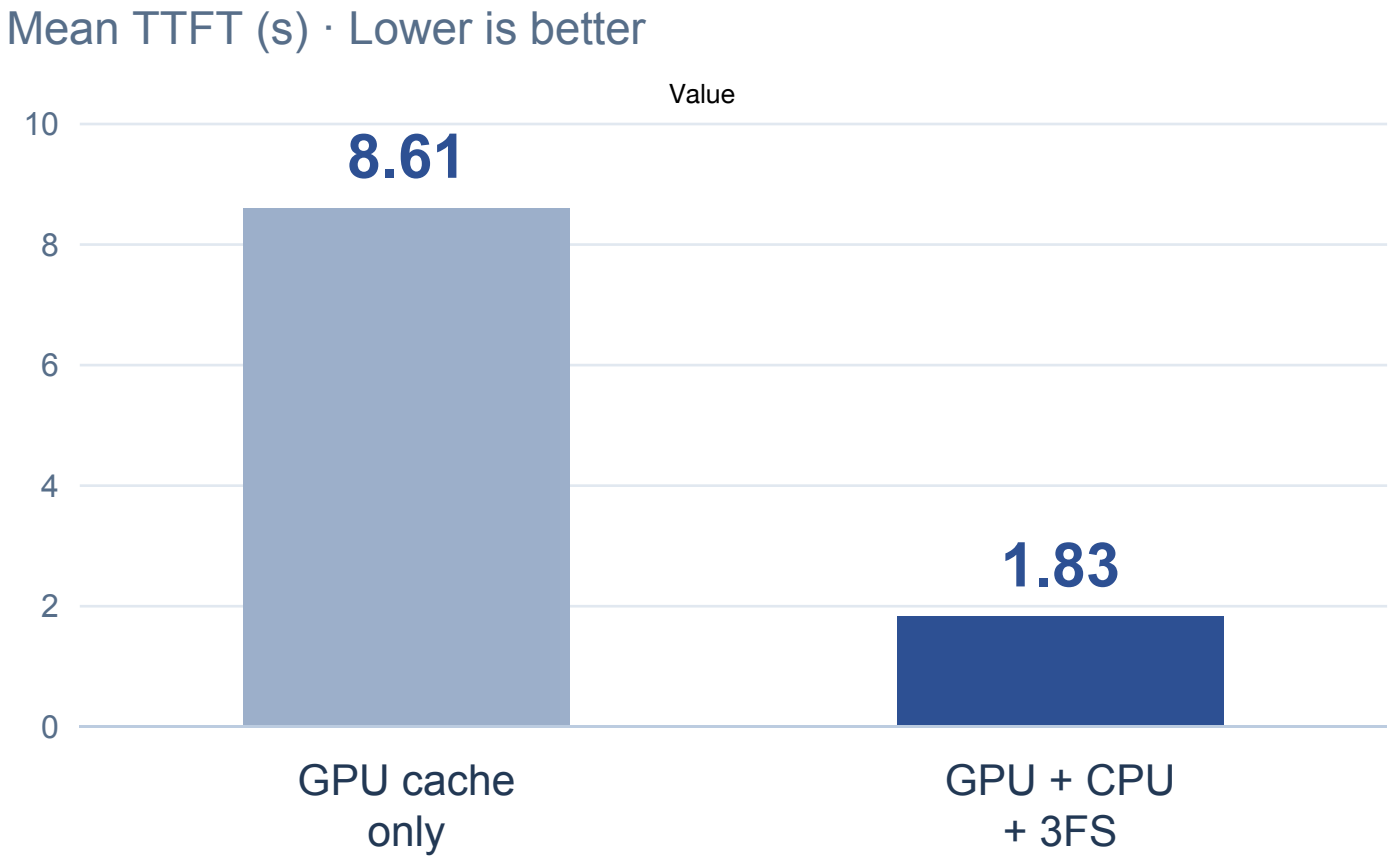
3 Load: G from host to GPU



Retain KV in lower tiers; restore it to the GPU as needed.

HiCache: Less Recomputation, Lower TTFT

DeepSeek-R1 · LooGLE · 8×H20-3e



78.7%

Lower mean TTFT

Source: HiCache long-context benchmark (2025)

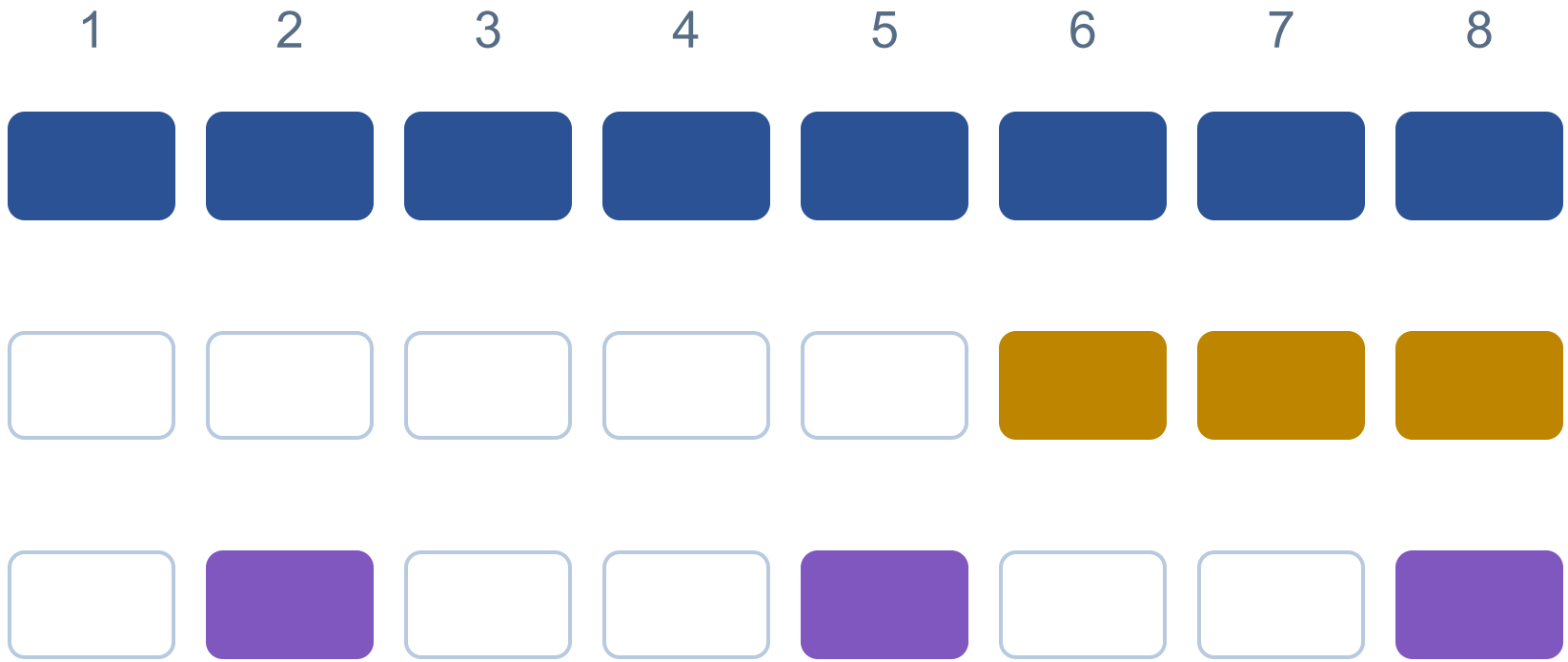
04

Unified Radix Tree



One Prefix, Three Types of State

MAMBA: resume only at checkpoints



Each component has its own validity rules

Previous Design: More Combinations, More Trees

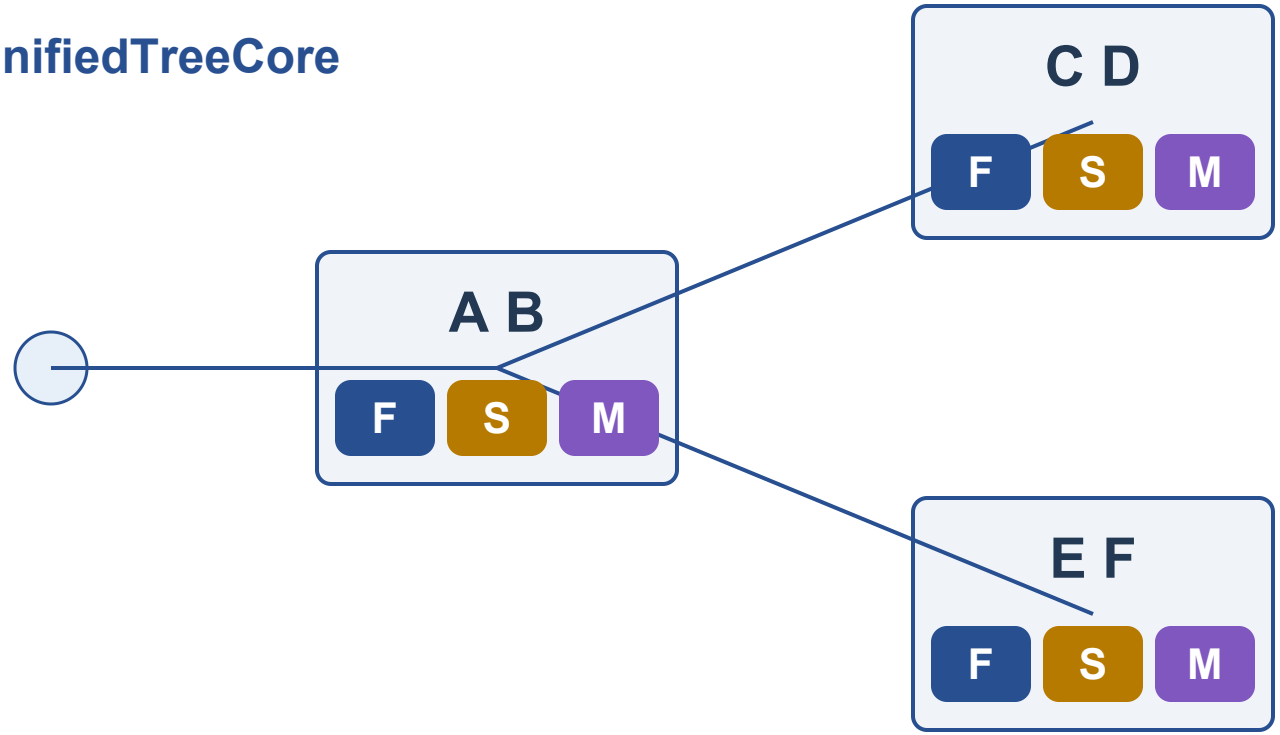
The same tree operations are maintained in multiple implementations

Implementation	File	Lines
RadixCache	radix_cache.py	885
SWARadixCache	swa_radix_cache.py	1,457
MambaRadixCache	mamba_radix_cache.py	1,443
HiRadixCache	hiradix_cache.py	2,021
PureSWARadixCache	pure_swa_radix_cache.py	153

match **split** **lock** **evict**

Unified Radix Tree: One Tree, Multiple Components

One token topology, independent state slots



F **FULL**

All KV

C128

sidecar

S **SWA**

Trailing window KV

M **MAMBA**

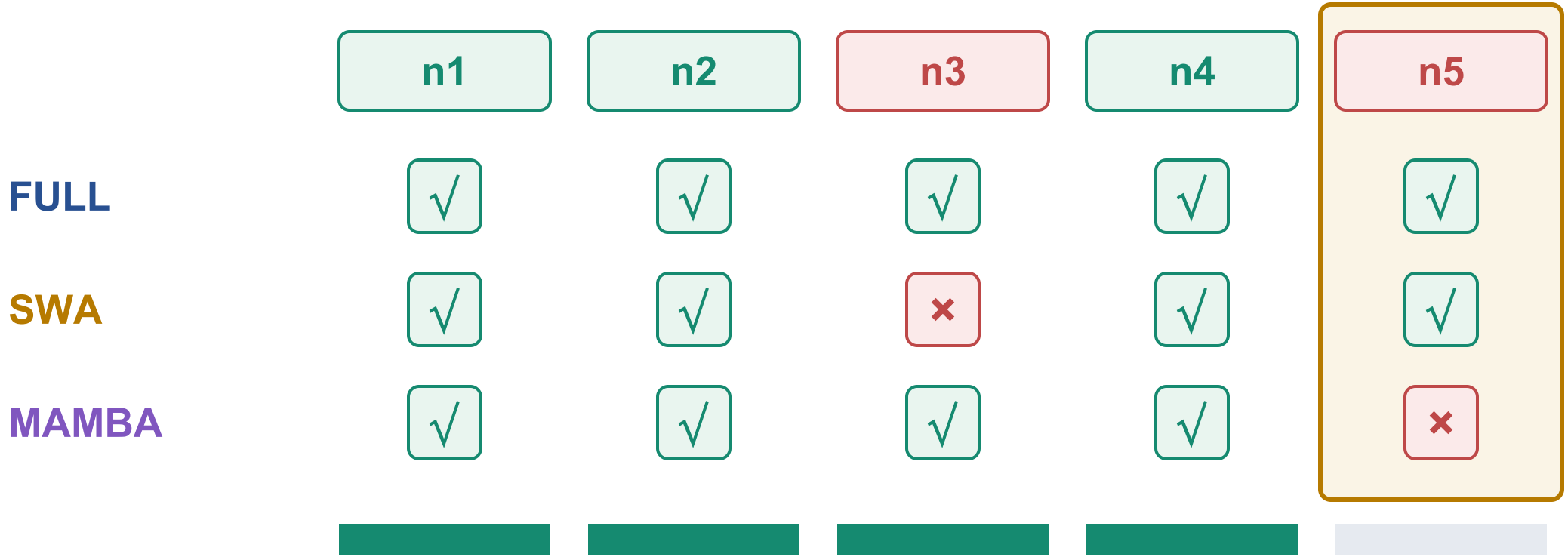
Boundary checkpoint

match · split · insert · lock · evict

Share nodes and tree operations; components define reuse rules.

Unified Radix Tree: Safe Reuse Boundaries

n5: no MAMBA checkpoint; reuse through n4



Deepest safe boundary: n4

Recompute n5

Keep checking even after a candidate is rejected.

Component Hooks: Core and Component Roles

The core handles tree operations; components define state rules

UnifiedTreeCore

Tree walk and node split
Component call order and pool orchestration

match
`match_prefix()`

split
`Tree node split`

insert
`begin_insert()`
`resume_insert()`
`end_insert()`

lock
`inc_lock_ref()`
`dec_lock_ref()`

evict
`evict_device_next_node()`
`evict_device_leaf()`

transfer
`demote()`
`drive_host_eviction()`

hooks →

- **FULL**
- **SWA**
- **MAMBA**

validator: reuse eligibility
split: partition component state
data: state and pool ownership
lock / eviction: component rules
PoolTransfer: cross-tier transfers

Other Optimization Based On Unified Radix Tree



Session-aware Eviction: Cache for the Next Turn

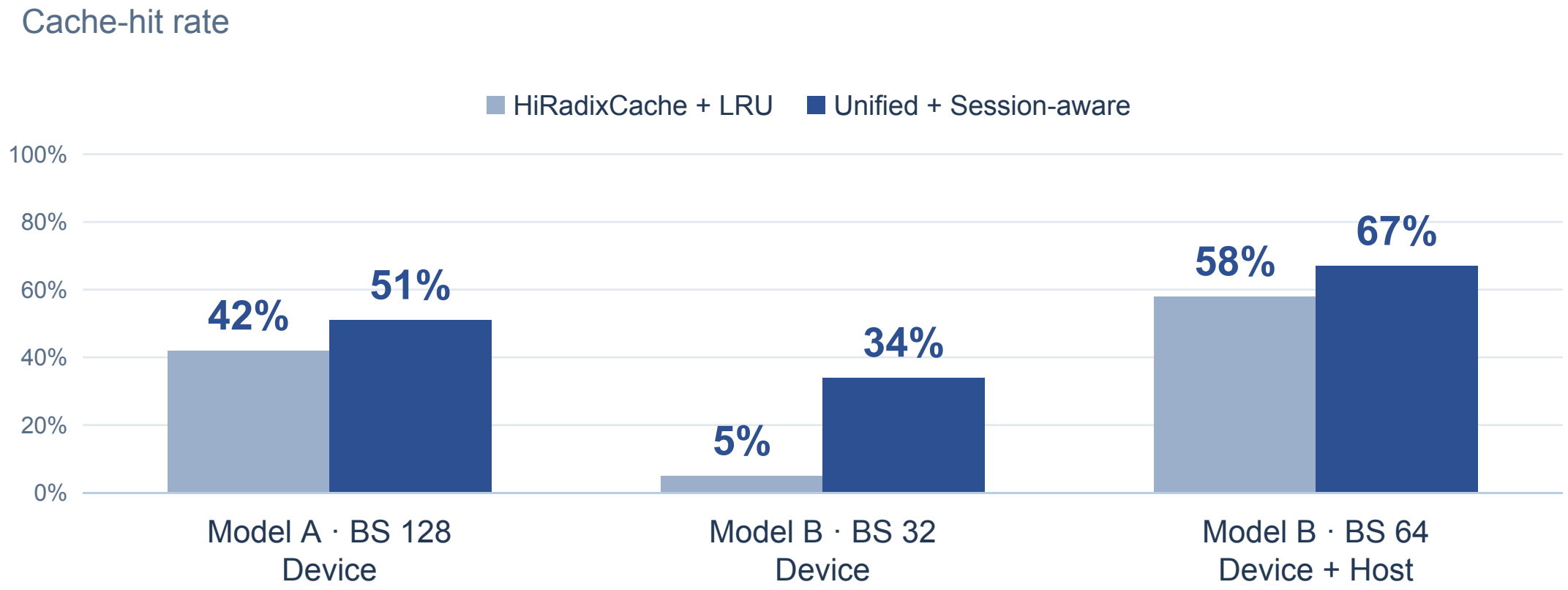
Prefer to evict entries with no active session references.

Least recent → Most recent

Cache entry	A1	B1	AB	C1	C2
Session refs	A	B	A + B	None	None
Traditional LRU	Evict	Evict	Keep	Keep	Keep
Session-aware	Keep	Keep	Keep	Evict	Evict

SWE-bench: Cache Configuration Comparison

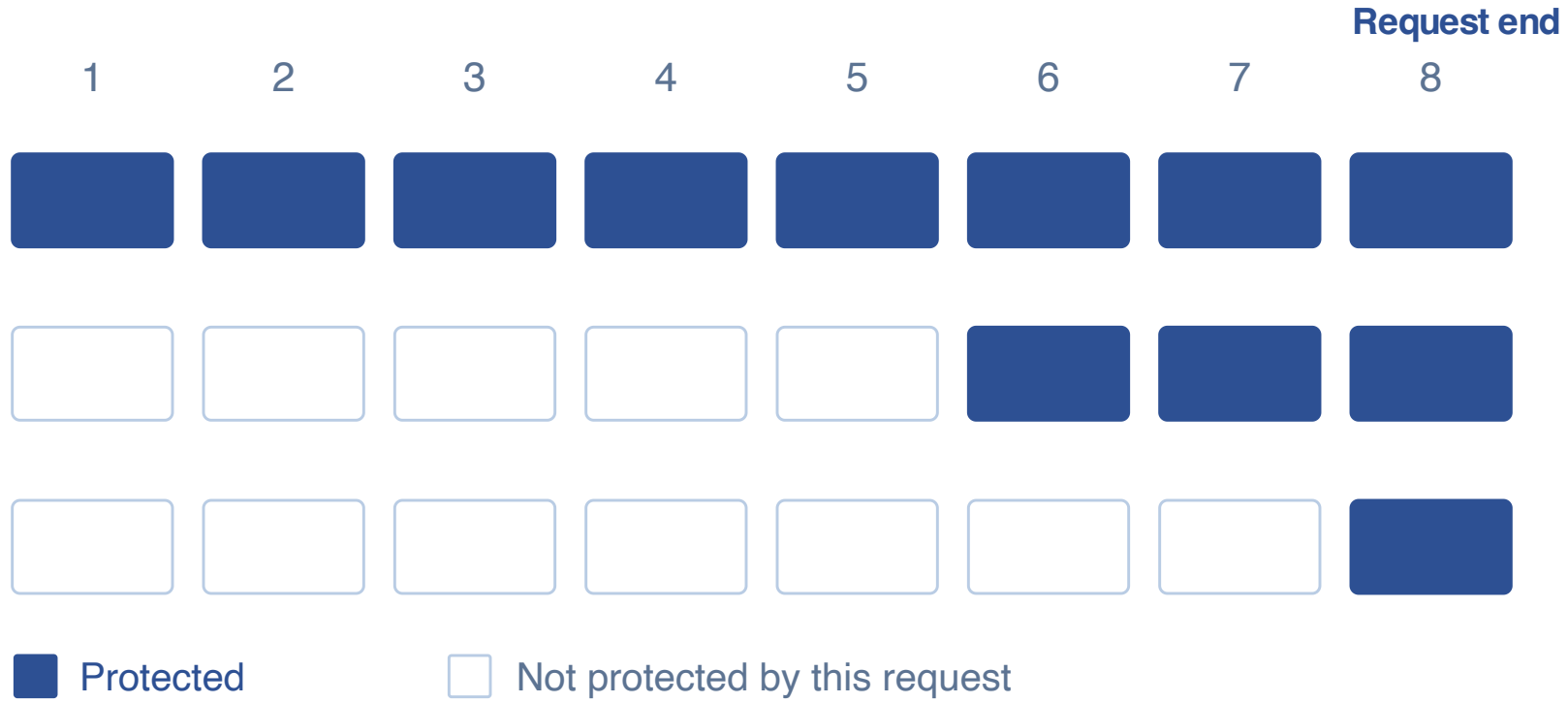
SWE-bench trace replay · Comparison of two cache configurations



Component Locks: Protect Only Needed State

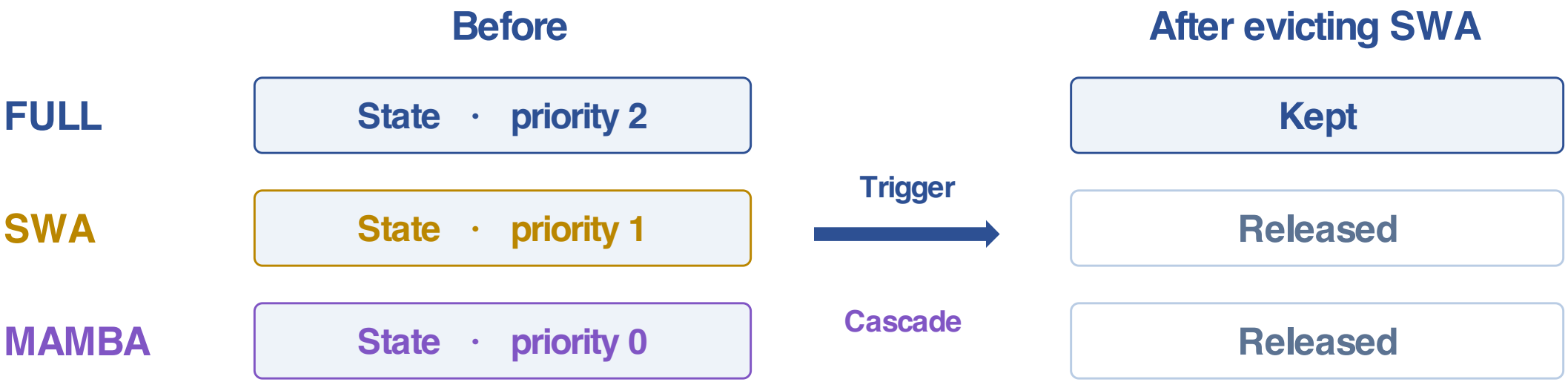
Each component locks only the state this request must keep.

Example: request ends at block 8; SWA window = 3 blocks.



Component Eviction: Priorities and Cascades

Cascade rules on one internal node with evictable state.



Assign retention priorities based on how likely eviction is to disrupt reuse at downstream nodes.

Branch Points: Keep State for the Next Fork

Actively keep the state needed to resume at a detected branch point.

Example: SWA window = 2

Shared prefix **Branch point P4**

1	2	3	4
Today	Alice	went	to

Saved at P4

FULL KV: “Today Alice went to”

SWA KV: “went to”

Mamba: checkpoint C4, if used

Existing continuation

Beijing	for	work

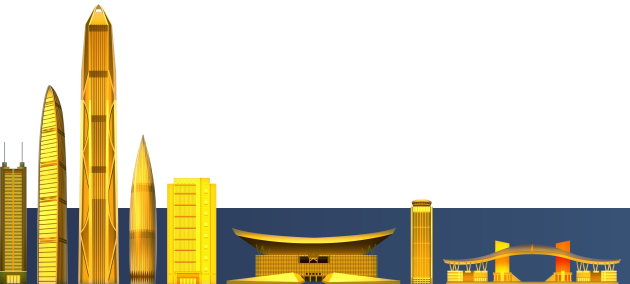
Current SWA window: “for work”

Later request: a new branch

Shanghai for a meeting

Reuse through P4.

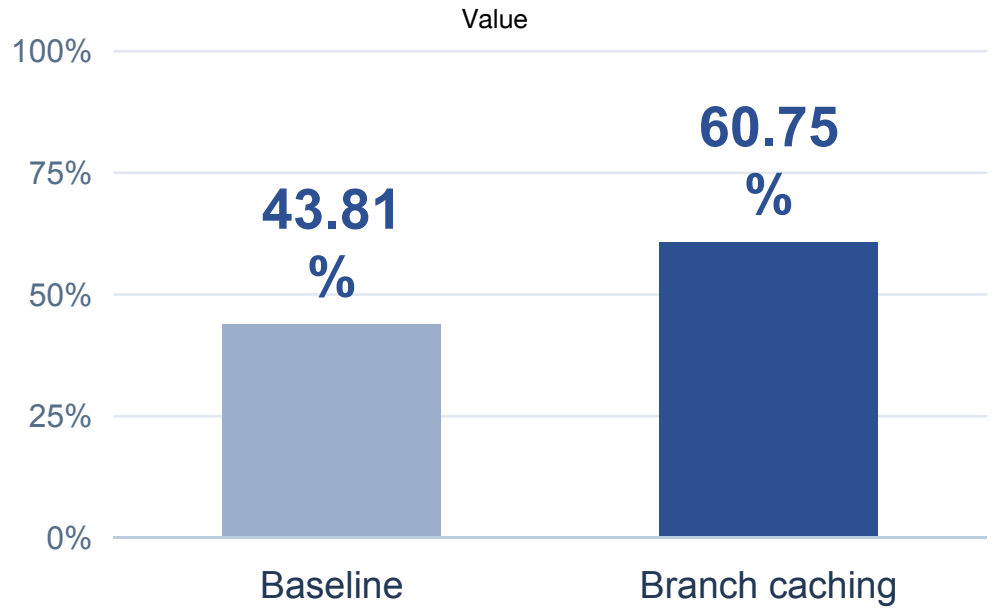
Compute the new suffix.



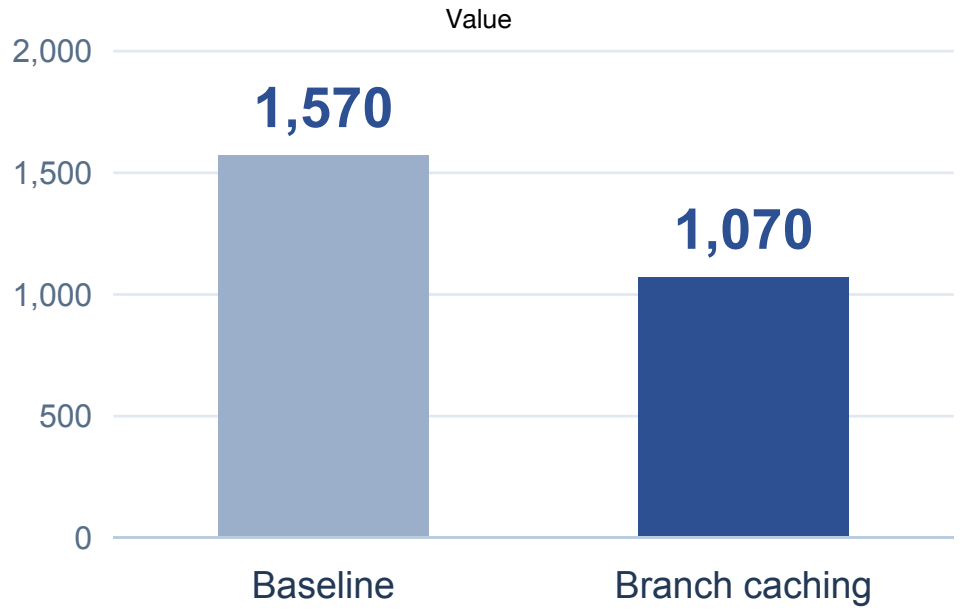
Branch-point Caching: Reuse at Forks

Shared-prefix benchmark · Out-of-window release enabled

Token hit rate



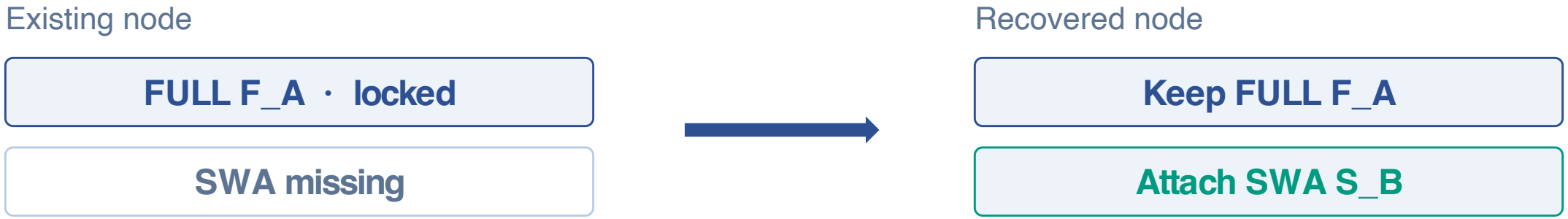
Mean TTFT (ms) · Lower is better



Cache Recovery: State and Read/Write Ordering

Merge recomputed state safely while other requests are running.

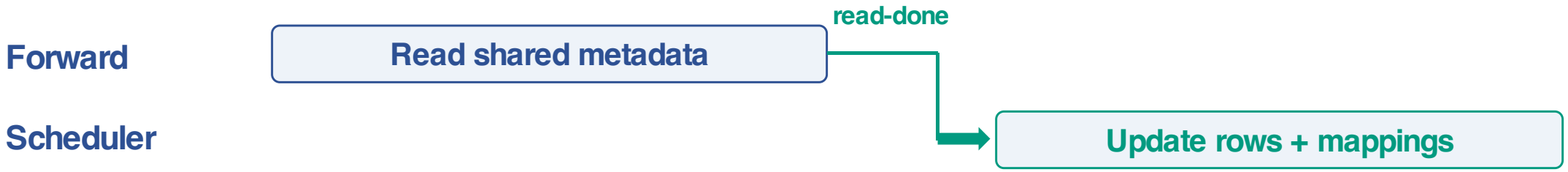
1 Attach recomputed SWA to an existing node



Incoming: FULL F_B + SWA S_B

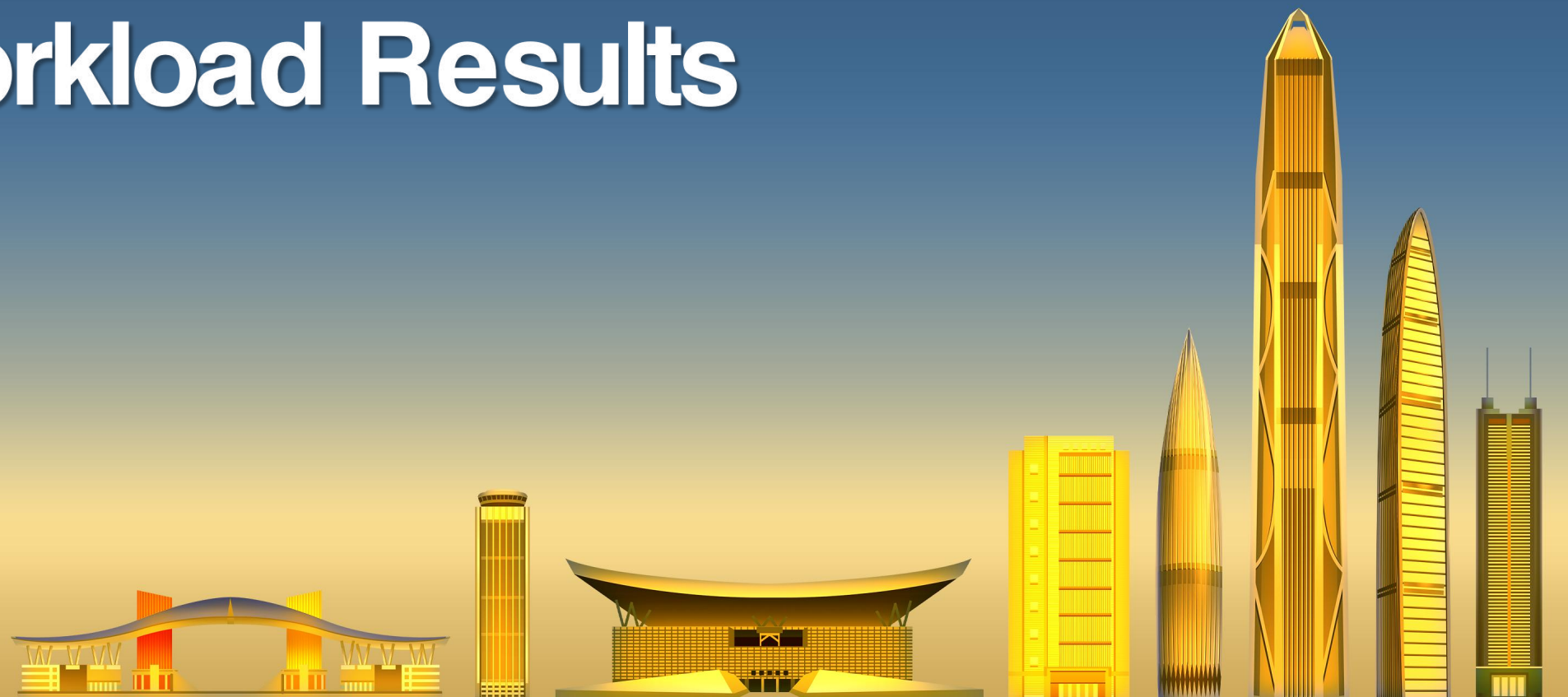
Map F_A → S_B; release duplicate F_B

2 Publish mapping updates after forward reads



05

Real-Workload Results

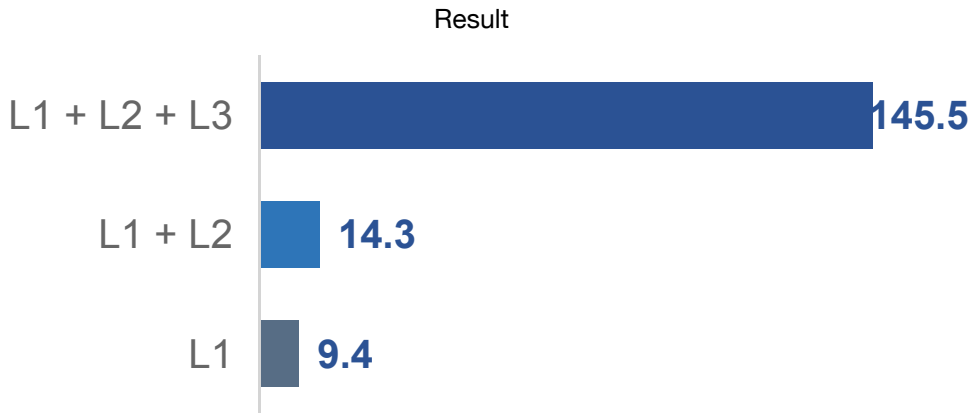


HiCache Results: Hybrid Models

Effective input throughput · k tokens/s (including cache hits)

FULL + SWA

DeepSeek-V4-Flash 4×H200 · 48 clients · 60 turns

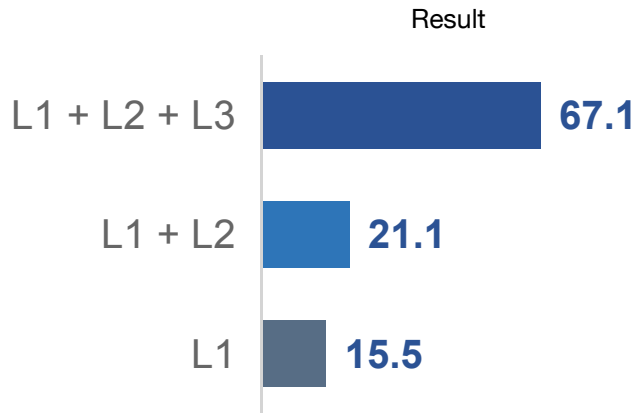


≈98%
Later-turn hit rate

< 9 s
Mean TTFT

FULL + SWA + MAMBA

Inkling-Small 8×H200 · 64 clients · 30 turns

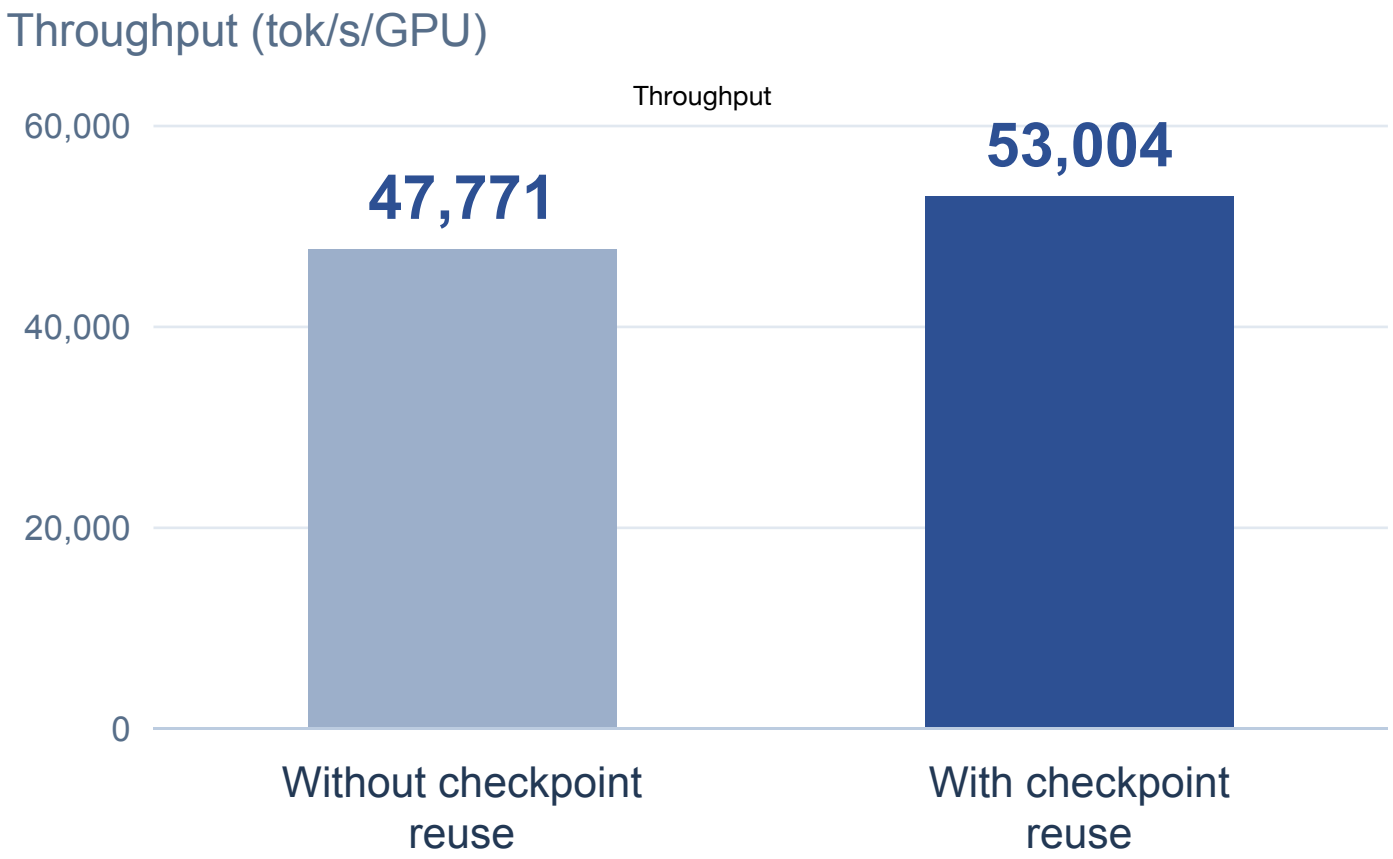


96.8%
Final hit rate

1.23 s
Mean TTFT

Independent Results: Recurrent Checkpoint Reuse

SemiAnalysis · Real agentic coding traces



+11%

Throughput gain

92.4%

Cache-hit rate

Next: Context Caching Across the Serving Stack

From per-worker caches to coordinated reuse across the serving stack

Router	Worker	HiCache
Choose where requests run	Execute and reuse state	Move KV across memory tiers

MERGED

Session / cache affinity
with load-aware fallback
#41990

Rust TreeCore is the default
for supported configurations
#39627

Mooncake external linker
for direct load-back
#37381

OPEN PRs

Count saved prefill in routing
Retry after admission rejection
#42430 / #42432

Torch-free cache simulation
Insert from a retained node
#38636

Share L3 keys across topologies
Prefetch L3 → L2 during tool calls
#39727 / #42434
Prefetch prototype: FULL, file backend, one worker.

Next: orchestrator hints and shared KV across workers

Coordinate sessions and tool calls with prefetch, demotion and retention.

Roadmap #21846 / RFC #27574 Status checked 4 Oct 2026

Three Takeaways

01 **RadixAttention**

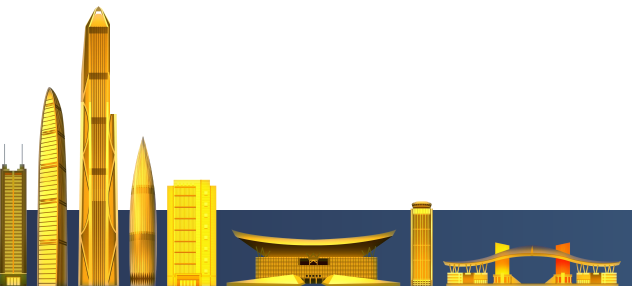
Match prefixes, reuse KV

02 **HiCache**

Store across tiers, restore on demand

03 **Unified Radix Tree**

Share the tree, compose state rules



GOSIM

GOSIM Shenzhen 2026

Thank You

